

GPU HA: Two-Tier Failover for LLM Inference — What DNS Can and Can't Do for You

Scott McDonald · v1.0 (2026-07-05) · Status: PUBLIC — released as an open reference implementation and prior art under Apache-2.0. The single-telemetry-contract mechanism (§8) is now described in full; nothing is held back.

Everything below describes built, tested, and drilled code unless explicitly marked. The two-tier telemetry contract is now proven across **three independent transport legs — localhost, LAN, and public internet**. Both of those placeholders are now closed: real-GPU failover *targets* are proven in Phase C'-a (intra-pool) and Phase C'-c (cross-cloud), and the multi-cloud orchestrator (gpuha up/down/reap) is built and proven live in Phase O. Everything through Phase O is done.

1. Why I built this

Twenty-some years ago I built dnshat.com — DNS-based failover for websites — at a time when people said DNS caching made it impossible. It worked, it was popular because it worked, and it led to an AWS acquihire. When GPU inference became the workload everyone's availability story depends on, the obvious question was whether the same trick works again: can DNS route AI traffic around dying GPUs?

The honest answer, which this project exists to demonstrate rather than assert, is: **half of it can**. The half that can't is the interesting half, and the boundary between them is precise enough to draw in code. GPU HA is the working proof — a two-tier failover system for LLM inference, built and drilled on real GPUs across three clouds (GCP, RunPod, Lambda), then published in full as open reference and prior art, as a learning vehicle for the modern inference stack (vLLM, multi-cloud GPU capacity, L7 streaming proxies) and a portfolio piece.

2. The thesis: where DNS's reach ends

Website failover suited DNS because the failure it routed around was **coarse and sticky** — an origin died, and new connections needed to land elsewhere within seconds-to-a-minute. Resolver caching hurt a little; it didn't matter much, because requests were short and stateless. A stale answer cost one failed page load and a retry.

LLM inference breaks that model in two specific ways:

The connection outlives the decision. A streaming completion holds a connection open for 5–90 seconds. DNS resolves once, at connection setup; the moment the socket opens, DNS is out of the loop. No TTL is low enough to re-route a request that is already in flight.

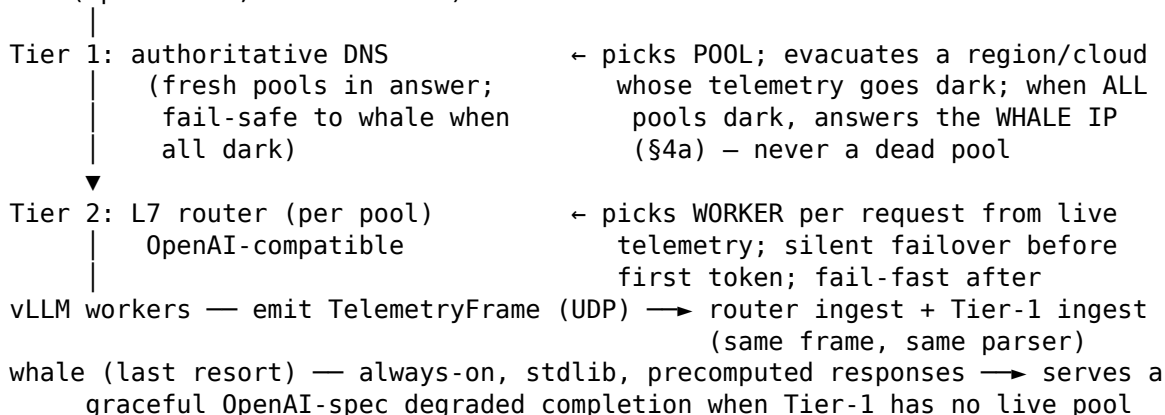
The decision is fast-moving and per-request. “Which node has the lowest time-to-first-token and the most VRAM headroom *right now*” changes second to second. Public resolvers override TTL 0 with 20–60s minimum caching, freezing a per-second decision for a minute — for every client behind that resolver.

So the design splits cleanly: **DNS (Tier 1) owns what it is unbeatable at** — coarse pool evacuation: a whole region or cloud losing capacity, telemetry going dark, cross-cloud disaster failover. **An L7 hop (Tier 2) owns the per-request decision** — which specific worker serves this request, and what happens when that worker dies mid-stream. The dnshat lesson wasn't “DNS solves everything”; it was “DNS solves that specific failure mode well.” The discipline is knowing which one you have.

3. Architecture

One versioned telemetry frame drives both tiers — that single contract is the load-bearing design decision (§8).

client (OpenAI SDK, one base URL)



Tier 2 components, all stdlib Python (a deliberate constraint — the code runs anywhere, and the failover logic stays legible rather than buried in framework config):

- **Selector** — health gate before scoring (a worker with stale telemetry is *dead*, never “idle and available”), transparent weighted scoring (TTFT and VRAM headroom dominate; price is an intra-pool tiebreaker), a per-worker circuit breaker, and anti-herding (squared-score sampling among the top N, so the best worker is preferred without being stamped).
- **Router** — one OpenAI-compatible endpoint; walks the selector’s retry plan; enforces the first-token boundary (§4); handles both chunked and Content-Length upstream framings.
- **Telemetry shim** — scrapes a *stock* vLLM server’s Prometheus /metrics, packs the numbers into the versioned frame, pushes UDP. The GPU box stays unmodified vendor software; all custom code lives on a cheap CPU box.

Tier 1 (`tier1_dns.py`, built and drilled in Phase D) is a minimal authoritative DNS responder that imports the **same ingest class** the router uses — shared code, not merely shared format. A pool appears in the answer iff at least one of its nodes has fresh telemetry within a deliberately coarser window (10s pool eviction vs. the router’s 3s node gate — evacuation should be slower-twitch than per-request selection); if every pool is dark, it fails **open** to a configured static-safe set — never an empty answer, never NXDOMAIN, never stale state served as live. That fail-open choice closes a question left open in the original design: when telemetry dies, a DNS tier that keeps confidently serving its last opinion is lying, and one that returns nothing takes the service down itself; serving the full known set and logging the degradation is the only honest option.

A lineage note: this project’s ancestor — a custom CoreDNS hidden master with a native Go plugin (`gpuha.go`), built in an earlier R&D iteration — was recovered during Phase L. Its telemetry model is the direct ancestor of the current design and its instructive *contrast*: a central aggregator daemon hardcoded fleet state and UDP-broadcast it to the master, which scored nodes and rewrote a zone file, firing RFC-1996 NOTIFY to five anycast slaves. Two things the ancestor makes concrete by contrast: (1) it did per-request “best node” selection *in DNS* — rewriting an A record to one winning IP per telemetry cycle — the exact responsibility the two-tier design deliberately moved to the L7 router; and (2) its liveness was a central aggregator’s `last_seen` stamp, a single point of failure, versus the POC’s consumer-judged arrival freshness off a versioned, content-free frame with no central aggregator at all. A small, almost poetic detail surfaced in recovery: the ancestor’s live zone file still resolved `api.gpuha.com` to a Lambda IP that no longer existed — the old engine’s *last computed routing*

decision, frozen in DNS, outliving the infrastructure it pointed at. DNS state is sticky and survives what it describes, which is, after all, half this project's thesis. The recovered source is preserved (privately) as prior-work reference; wiring a native TelemetryFrame parser into CoreDNS is deferred production polish (D2). The POC's Tier-1 deliberately re-proves the mechanism in ~200 lines of the same stdlib Python as the rest of the stack, importing the *same ingest class the router uses*.

4. The first-token boundary — the honest failover contract

The central claim of any streaming-inference HA system has to answer: *what exactly happens when a worker dies mid-generation?* Most marketing hand-waves this. The truthful contract is:

- **Failure before the first token reaches the client:** fully recoverable. The router silently retries the next-best worker; the client sees nothing but slightly higher TTFT.
- **Failure after the first token:** *not* transparently recoverable. Tokens already streamed came from one worker's KV-cache state; no other worker shares it. Pretending to resume would mean silently regenerating different output. The honest behavior is a clean truncation and a client-level retry of the whole request.

The router enforces this as control flow — a committed latch flips the instant the first content byte is forwarded, and no code path retries after it. In the Phase A drills (§6) both sides of the contract were exercised against a real GPU being killed.

(Production note: shared/disaggregated KV-cache designs could move this boundary. That's a research-grade extension, out of POC scope, and worth naming precisely *because* the current contract is honest about it.)

4a. The fail-whale — graceful degradation when everything is dark

The first-token contract governs *a* pool with *some* live worker. But what does a client get when **every** pool is dark — all GPUs gone, all telemetry silent? The naive answer is what Tier-1's fail-open originally did: keep the pools in the DNS answer, and let clients hit dead router IPs. Honest at the DNS layer, broken at the client — connection-refused and SDK stack traces.

The fix borrows a pattern from the dnshat era. When Twitter couldn't serve traffic, DNS failed over to a static "fail whale" page — the user saw something human, not a browser error. Re-built for OpenAI-spec token traffic: a last-resort endpoint (`whale.py`) that speaks the protocol and answers when nothing else can. Tier-1's FAILSAFE now points at the whale IP instead of at dead pools. The whale serves either a **protocol-correct 503 + Retry-After** (which official SDKs auto-retry, the self-healing path for API clients) or, for chat apps, a **valid 200 streamed completion** whose content is a polite "give me a moment, I'm having resource trouble" message — signaled in-band via `"model": "gpuha-degraded"` so naive frontends render the text while careful clients detect degradation from a field they already parse.

Governing design principle: **the whale is the dumbest thing in the fleet**. No telemetry, no state, no per-request work, no dependencies — every response is a byte buffer precomputed at startup. A last resort that shares fate with the things it's a last resort *for* is decoration. Measured ~5k req/s single-process (stdlib only); scales out via `SO_REUSEPORT`.

This was proven end-to-end across the public internet in Phase D.5 (§6): with both pools killed, a real external client followed DNS to the whale IP and received HTTP 200, `"model": "gpuha-degraded"`, and the graceful message — never a connection error. The complete failure arc — pools dark → DNS evacuates → DNS fails to whale → client gets a courteous OpenAI-spec answer — runs on real infrastructure.

The honest residual (same one dnshat had): clients holding a cached DNS answer (public-resolver TTL floors of 20–60s) still hit dead pool IPs until their cache expires. The whale covers the post-TTL experience and the router-local “my pool is dead but I’m alive” case; a fully dead pool host answers nothing until TTL expiry. Anycasting the whale IP is the deferred production mitigation. The whale does not claim to close the cached-DNS window — it closes everything downstream of it.

5. Liveness is the consumer’s judgment: silence = death

The single most important HA decision in the system is negative-space: **no worker ever asserts “I am healthy.”** Frames carry raw metrics plus identity and sequence; each consumer judges liveness by *its own clock* against frame arrival. A node that stops emitting ages out of the eligible set within seconds — no error required, no health-check endpoint to lie.

This paid for itself immediately in drills: a killed vLLM leaves an orphaned subprocess holding all GPU memory (§7.3). A “is the process up / is the port open” health check misreads that state; arrival-freshness cannot be fooled by it, because the shim’s scrape fails and emission simply stops.

Two supporting invariants that turned out to matter: the emitter’s scrape timeout must sit well below the consumer’s freshness window (a scrape that can block for 3s against a 3s window makes a merely *slow* worker flap in and out of eligibility — slow ≠ dead; slowness is what the TTFT score is for), and breaker-cleared and telemetry-fresh are independent gates — a recovered worker needs both.

6. Drill results

Phase A — Tier-2 failover against a real GPU (RunPod, L4, vLLM 0.11)

Drill	Result
Pre-token failover: kill -9 vLLM during live traffic	5/5 requests completed with 200s, transparently served by fallback workers; no client-visible error
Silence pruning: telemetry stops	worker dropped from eligible set within the freshness window, no request had to fail first
Mid-stream fail-fast: kill during generation, post-commit	client stream cleanly truncated (no fake resume, no counterfeit [DONE]); fail-fast recorded
Recovery: restart vLLM	worker re-entered eligible only after breaker cooldown AND fresh telemetry — both gates

Honesty note: the *dying* worker was a real L4; the failover *targets* were protocol-identical stubs, pending multi-GPU quota. This asterisk is now removed – see Phase C’-a below, where the failover target is itself a real GPU-backed worker.

Phase D / D.5 — Tier-1 evacuation, proven across three transport legs

The Tier-1 mechanism was proven in three stages, each adding a harder transport reality: **localhost** (single host, two simulated pools), **LAN**, and finally **public internet** — a Seattle node firing TelemetryFrames to a Miami plane. The cross-internet drill (D.5) is the one that matters, because it is the first time frames crossed NAT, real peering, and firewall paths rather than a loopback:

State	dig api.gpuha.com returns	tier1 log	dropped_frames
Baseline (both pools fresh)	plane IP + remote-pool IP	ANSWER SET -> [plane, linode-west]	{}
Evacuate (remote emitter killed)	plane IP only	ANSWER SET -> [plane]	{}
Restore (remote emitter back)	both again	ANSWER SET -> [plane, linode-west]	{}
FAILSAFE (both pools dark)	whale IP alone	[FAILSAFE->WHALE <plane-ip>]	{'linode-west': 185}

Two results worth dwelling on. First, **zero UDP loss on the live cross-internet path** (dropped_frames={} through baseline/evacuate/restore) — a clean transport result over real public internet, not loopback. Second, and more telling: the seq-gap count of **185 appears only at FAILSAFE**, i.e. only after the remote node went silent. That is the freshness gate proving itself with a number attached — silence registered as accumulated sequence gaps, the node aged out, and the system degraded to the whale instead of continuing to advertise a dead pool. “Silence = death” is not a slogan here; it is an observable count.

Final proof, from the remote node acting as an external client that followed DNS to the whale: curl returned HTTP 200, "model": "gpuha-degraded", and the graceful message. A real cross-internet client, with every backend dead, received a courteous OpenAI-spec answer rather than a connection failure. That closes the distributed proof.

Honesty note: these drills used lightweight stub emitters as the *pools* (the point of D.5 was the DNS/transport/whale layer, which is content-agnostic — it evacuates on telemetry silence and never inspects inference). Real GPU *pools* on both ends, and intra-pool GPU-to-GPU failover as the evacuation target, is now proven – see Phase C'-a (intra-pool GPU-to-GPU) and Phase C'-c (cross-cloud, real GPU pool) below. What is proven: the full Tier-1 evacuation + whale mechanism, over real public-internet transport, with real DNS queries and the shared ingest class.

Phase C'-a - intra-pool real-GPU failover (Tier-2 L7 router)

This removes the last stub-asterisk from Phase A: the failover *target* is now a real GPU-backed vLLM worker, not a protocol stub. Setup: ONE Lambda A10 running TWO real vLLM workers (Qwen2.5-3B; memory-split via gpu-mem-util 0.38, max-model-len 2048, max-num-seqs 8; ports 8000/8001) behind ONE router (backends gpuha-w1 and gpuha-w2), one shim per worker.

Phase	Router eligible set	Requests served by
Before kill	[gpuha-w1, gpuha-w2]	BOTH workers, real tokens
Kill gpuha-w1	pkill -9 vLLM on port 8000	(w1 goes dark)
After kill	[gpuha-w2] only	gpuha-w2, HTTP 200, real tokens

w1 was evicted on telemetry silence; every subsequent completion returned X-GPUHA-Served-By: gpuha-w2, HTTP 200, and real tokens (“GPU HA online”) with zero client-visible errors. Mid-stream fail-fast is handled by the first-token contract. Two vLLM co-resident on one physical A10 is a cost/reliability compromise; the failover *mechanism* and a real-GPU-backed *target* are proven, and two physically separate GPUs is the trivial extension (two instances wired to one router, or a 2-GPU box). Evidence: docs/evidence/cprime-a/.

Phase C'-c - combined multi-cloud demo, orchestrator-driven

The whole topology stood up from a single command. `gpuha up demo-full` launched a real Lambda A10 GPU pool (`gpuha-target-lambda`) and a GCP pool (`gpuha-target-gcp-stub`) with `min_pools: 1`, and a dead-man's switch was armed as a cost backstop. Everything below ran over real public-internet transport.

Drill	Action	<code>dig api.gpuha.com</code> result
Baseline	both pools fresh	ANSWER SET -> [GCP, Lambda]
Cross-cloud evacuate	kill the GCP pool	ANSWER SET -> [Lambda] only
Whale finale	kill the Lambda shim	FAILSAFE -> WHALE (single whale IP)

After the cross-cloud evacuation a real completion *still flowed* through the surviving Lambda pool (X-GPUHA-Served-By: `gpuha-w1`, HTTP 200) – the DNS answer set collapsed to one cloud without interrupting live inference. At the whale finale, with both pools dark, `curl` to the whale returned HTTP 200 and the graceful `{"model": "gpuha-degraded"}` message rather than a connection failure.

Teardown is the honest part: `gpuha down` threw a `NoneType ... has no attribute 'strip'` error on the GCP-stub delete path, and the `reap --all` backstop caught and terminated the Lambda instance anyway – `teardown-first` design doing exactly its job. Final spend returned to zero. Evidence: [docs/evidence/cprimec/](#).

7. What reality taught us that the tests didn't

The unit and integration suites (five simulated scenarios, five telemetry-contract tests) caught real design bugs before any cloud spend — notably that linear score-weighting washes out small quality gaps into coin-flips (fixed by squaring), and the breaker/freshness gate independence. But four bugs only surfaced against real infrastructure, and they're the most instructive part of the project:

7.1 The sequence lockout. Frames carry a monotonic per-node seq; the ingest rejects `seq <= last_seq` to defend against UDP reordering. A *restarted* emitter resets seq to 0 — and is therefore locked out forever, indistinguishable from replayed packets. Reality: every emitter eventually restarts. POC fix: seed seq from wall-clock. Principled fix (frame v2): an epoch/boot-id field so the ingest can distinguish “reordered datagram” from “restarted emitter.”

7.2 Framing dispatch. The router originally assumed chunked transfer-encoding, because the test doubles always streamed. Real vLLM only chunks when the request asks for `stream: true`; buffered responses arrive with Content-Length, which the chunk parser read as garbage — producing *false breaker trips* against a perfectly healthy worker. An HA layer whose own parsing bug marks healthy workers dead is a self-inflicted outage. Lesson: test doubles inherit your assumptions; the real dependency is the test.

7.3 VRAM zombies. `kill -9` on vLLM orphans its EngineCore subprocess, which keeps holding ~all GPU memory — the *node* looks restartable while the GPU is occupied, and the restart fails on memory. This is a genuine GPU-HA failure class: process liveness and resource liveness diverge. Silence-based liveness handled it correctly (§5); a telemetry-level zombie signature (VRAM ~full with no serving process) is logged as a frame-v2 detection note.

7.5 The infrastructure fought us harder than the code did. A recurring, humbling pattern: more time was lost to *transport and tooling* than to the actual distributed-systems problem. A browser-driving agent kept reaching for browser-shaped doors — driving a serial

console by screenshot, pasting files as base64 chunks through a terminal — when a real shell or an upload button was right there. Cloud Shell silently blocked outbound UDP (invisible until it broke a transport test) and disconnected on long operations. A stopped Linode still bills (unlike AWS/GCP), quietly draining ~\$68/mo across five forgotten boxes. A custom-named SSH deploy key isn't offered by default, producing "the key is definitely on the repo but still permission denied" until an `~/.ssh/config IdentityFile` entry fixes it. None of these are glamorous, and all of them cost real hours. The lesson worth generalizing: for a project whose entire premise is operational resilience, the operational friction is the curriculum — the scars became a per-provider lifecycle matrix, and that matrix becomes product code.

7.6 An agentic build workflow, honestly assessed. This was built with a three-role split: a human supervisor (decisions, approvals, credentials), a reasoning model as architect (design, briefs, code artifacts, review), and a browser-capable agent as engineer (drives consoles/UIs, runs commands). Briefs are the interface; anything irreversible (deletions, spend) is human-gated; credentials never pass through an agent (keys generated by the human, public halves only; root passwords typed by the human). What worked: the split kept the architect out of infra and the engineer out of redesign, and the agent's best moment was refusing to claim a capability it didn't have — told to "just SSH in," it *tested*, found its sandbox had no egress and never held the private key, and said so, instead of pretending. What didn't: browser agents default to browser-shaped solutions even when worse; the "bootstrapping gap" on a fresh box (no shell an agent can reach until you stand one up) is inherently the human's to cross; and version control is the only real cure for the divergent-copies problem that otherwise metastasizes across sandbox, connected folder, and box. Verify-don't-claim is the rule that most improved outcomes — for the models and the human alike.

7b. Cold-start time is a first-class HA cost - and even the mitigation fights infrastructure I/O

The slowest part of standing up a replacement pool is not our code - it is provisioning. A cold GPU pod spends ~20 min installing the torch+vLLM stack; even the mitigation we measured (pre-baking that stack + model onto a network volume) still hit ~9 min mount-to-first-token, because vLLM reads ~22GB of libraries and weights over a network filesystem. The lesson: a warm tier (pre-baked images; hot/warm/cold readiness with per-tier SLOs) is a PRODUCT feature, not a POC fix. The POC's honest move is to measure cold-start truthfully and PARALLELIZE around it - acquire and bootstrap all pools concurrently so wall-clock equals the slowest pool - and to treat a 20-minute-to-recover pool as DR, not HA.

7c. The map diverges from the territory at the provider-default layer

The sharpest instance of this project's thesis showed up not in our code but in a cloud provider's defaults. In the final three-cloud run the Lambda pool registered healthy in Tier-1 - its telemetry reached the plane, DNS carried its IP - yet could not actually serve a completion: Lambda now blocks inbound on the router port by default. The router was alive and serving on the box (ss showed it listening; local curls returned tokens), but the plane could not reach it. SSH and outbound telemetry worked, so every health signal we trust said "up" while the one thing a client needs - a reachable completion - was firewalled off. Two things we assumed were identical, "the pool reports live" and "the pool can serve a request," were different, and only a real cross-provider run surfaced the gap. Same shape as the VRAM-zombie (process != resource liveness), the kill-test (state != provider truth), and the grey-failure (refused != silent): the mock never diverges from itself; the territory does.

8. The single telemetry contract (*sensitive section — held pending patent/open-source decision*)

[Placeholder: one versioned frame, two routing layers. The frame carries identity (node + pool), sequence, and raw metrics — never self-asserted health. The router consumes it for per-request worker selection; the DNS tier consumes it — same parser, same ingest class, literally shared code — for pool evacuation. Liveness in both tiers is consumer-side arrival freshness. Detail level here to be decided after counsel review if the patent path is pursued; otherwise expand fully.]

9. Limitations, stated plainly

- **Cross-cloud evacuation is proven at the DNS/telemetry layer, not - in the final run - via a live Lambda completion handoff.** Killing the RunPod pool made Tier-1 drop it from the answer within the pool window, leaving the Lambda survivor; going fully dark returned the whale, which served a degraded completion. What we did not show end-to-end that run was a live token stream failing over onto Lambda, because Lambda's default inbound firewall blocked the router port from the plane. That is a provider-configuration detail (add an inbound rule), not a property of the GPU HA design: the evacuation logic runs on telemetry, which flowed correctly, and the same handoff was shown live in earlier single-provider drills. Stated plainly rather than papered over.

Tier-1 evacuation, the whale, and the full failure arc are proven across localhost/LAN/public-internet transport; what remains is that the drill *pools* were stub emitters, not real GPU pools with intra-pool GPU-to-GPU failover as the evacuation target (now removed - proven in Phase C'-a / C'-c). Similarly, Phase A proved a real GPU *dying* but with protocol-identical stubs as failover *targets* (quota-bound to one real GPU) (now removed - proven in Phase C'-a / C'-c). Production DNS wiring — how `api.gpuha.com` resolves on port 53, authoritative-vs-hosted-zone, anycast slaves, native CoreDNS TelemetryFrame parsing — is deferred **[D2]**. Telemetry has no auth beyond source-IP scoping (frame v2: HMAC — matters more now that frames demonstrably cross the internet). No mid-stream resume (by design — §4). Scoring weights are hand-set, not learned. The multi-cloud orchestrator that spins the whole topology up on demand and tears it to zero — the actual product — is **[PHASE O]**, now built and proven live: the `gpuha` orchestrator runs up/down/reap against real GCP and Lambda GPU pools (Milestones 1 and 2), and drove the Phase C'-c demo end to end.

10. Production path

CoreDNS hidden-master wiring (the `dnshat`-lineage engine) with native frame parsing; anycast slave layer; 4+ real GPUs per pool across zones; frame v2 (epoch/boot-id, HMAC, zombie signature); Envoy-grade L7 (the custom router names its own lineage: the breaker is outlier detection in miniature, the selector is a custom EDS); learned scoring weights against real TTFT distributions.

Appendix A — Environment landmines (verbatim, they cost hours)

RunPod CUDA-12.4-era driver requires pinning `vLLM 0.11.0 + torch 2.8.0+cu128 + transformers<5 + hf_transfer` (default pip pulls a CUDA-13 build that won't run). GCE stop/start can change internal IPs — reserve static addresses for anything referenced by config. `vLLM` does not auto-start on boot from a manual launch. GCP capacity reservations bill at full rate while idle. EngineCore orphan cleanup: `nvidia-smi --query-compute-apps=pid | xargs kill -9` before `vLLM` restart. Fake-worker port 8001 collides with RunPod pod `nginx` — use 8011+. Lambda has no stop state — instances bill per-minute until *terminated*, and terminate destroys the local disk (commit/push before teardown). Stopped Linodes still bill the full plan

rate — delete, don't stop, for throwaway nodes. LISH/Weblish is an out-of-band serial console that always prompts for the root password and ignores SSH — it's a last-resort recovery door, not a working shell. Cloud Shell blocks outbound non-DNS UDP and disconnects on long operations — never run a workload in it; use it only as a control surface to drive a real instance. A custom-named SSH deploy key isn't offered by default — add an `~/.ssh/config` `IdentityFile` entry (check this first when “the key is on the repo but still permission denied”).

Current state (2026-07-05)

Published. GPU HA is released in full as an open reference implementation and prior art: the public repository (github.com/gpuha/gpuha, Apache-2.0) carries the code, this whitepaper, and a build-your-own guide, tagged `v1.0.0` with the whitepaper PDF attached. All demonstration and GPU infrastructure has since been torn down to \$0 — the always-on control plane that hosted Tier-1 and the whale was decommissioned at launch, provider GPU resources were reaped across GCP/Lambda/RunPod, and the `api.gpuha.com` demo record was retired. Nothing needs to keep running for the contribution to stand: the two-tier telemetry contract was proven across localhost, LAN, and public-internet transport, and the full stack is durable in git.

The environment is reproducible rather than resident: a one-shot bootstrap script rebuilds a hardened working box from bare Ubuntu, and the Tier-1/whale services redeploy from the published code — so the live demo is a short rebuild away rather than a standing monthly cost. Total project spend: a few dollars of GPU time; the five recovered legacy boxes (~\$68/mo) were deleted, so cleanup more than paid for the build.

Next: a *single combined* recorded demo standing up the full multi-provider topology at once with every completion path live (each piece is proven; the choreography and one Lambda firewall rule are the remaining step); **[D2]** production DNS wiring; and the deferred hardening (control-plane HA, frame v2, cold-start optimization). See `TARGET_STATE.md`.